

Politechnika Świętokrzyska w Kielcach

Wydział Elektrotechniki, Automatyki i Informatyki



Politechnika Świętokrzyska
Kielce University of Technology

Aplikacje mobilne

Projekt

Aplikacja mobilna - wypożyczalnia samochodów (React Native / Expo)

Skład zespołu:

Wiktor Niedopytalski

Kamil Grabiec

Kierunek/specjalność: Informatyka / Systemy Informatyczne

Studia: niestacjonarne

Numer grupy: 11A

Spis Treści

1. Wprowadzenie / Cel laboratorium.....	3
1.1. Krótki opis aplikacji.....	3
1.2. Wykorzystane technologie oraz narzędzia.....	3
2. Implementacja.....	4
2.1. Opis głównych funkcjonalności aplikacji.....	4
2.2. Prezentacja zrzutów ekranu.....	5
2.3. Wybrane fragmenty kodu z kluczowymi funkcjonalnościami.....	8
3. Testy.....	9
3.1. Opis metod testowania.....	9
3.2. Wyniki testów, napotkane błędy oraz zastosowane rozwiązania.....	10
4. Podsumowanie.....	11
4.1. Wnioski z realizacji projektu.....	11
4.2. Ocena osiągniętych rezultatów i refleksje na temat procesu implementacji.....	11
4.3. Propozycje usprawnień lub dalszego rozwoju aplikacji.....	11
5. Podział pracy.....	11
6. Literatura.....	12

1. Wprowadzenie / Cel laboratorium

Celem projektu było zaprojektowanie i zaimplementowanie kompletnej aplikacji mobilnej, która działa na urządzeniach z systemami Android i pokazuje, że potrafimy połączyć ze sobą najważniejsze elementy współczesnego frameworka mobilnego.

Założenia, które przyjęliśmy na początku, wynikały częściowo z wymagań przedmiotu. Aplikacja miała zawierać co najmniej dwa rodzaje nawigacji, korzystać z dwóch czujników urządzenia oraz wymieniać dane z zewnętrznym API z użyciem metod GET, POST i PUT. Wokół tych wymagań zbudowaliśmy wypożyczalnię samochodów, dzięki czemu poszczególne funkcje układają się w logiczną całość.

1.1. Krótki opis aplikacji

Aplikacja jest mobilnym katalogiem wypożyczalni samochodów. Po uruchomieniu użytkownik trafia na listę dostępnych aut pobieraną z serwera. Każdy samochód można otworzyć, żeby zobaczyć szczegóły (markę, model, rok produkcji, cenę za dobę oraz opis), a następnie przejść do formularza rezerwacji. W formularzu podaje się zakres dat, a aplikacja sama wylicza liczbę dni i łączny koszt wynajmu, po czym zapisuje rezerwację na serwerze.

Drugą częścią aplikacji jest zarządzanie własnymi rezerwacjami. Na osobnej zakładce widać listę wszystkich rezerwacji wraz z ich statusem (aktywna lub anulowana), a po wejściu w szczegóły można rezerwację anulować. Trzecia zakładka to profil użytkownika można w nim ustawić imię, e-mail oraz zdjęcie zrobione aparatem, a także pobrać aktualną lokalizację GPS i otworzyć ją w aplikacji map.

Warto dodać, że dane profilu celowo przechowujemy wyłącznie lokalnie, na urządzeniu, i nie wysyłamy ich na serwer. Chcieliśmy w ten sposób pokazać różnicę między danymi „chmurowymi” (auta i rezerwacje) a danymi prywatnymi użytkownika.

1.2. Wykorzystane technologie oraz narzędzia

Projekt powstał w oparciu o **React Native** w wersji dostarczanej przez platformę **Expo (SDK 54)**. Całość napisana jest w języku **TypeScript** z włączonym trybem strict, co wymusiło na nas porządne typowanie danych i wyłapało sporo pomyłek już na etapie pisania kodu.

Najważniejsze technologie i biblioteki, z których korzystaliśmy:

- **Expo Router** - routing oparty o strukturę plików w katalogu app/. To na nim zbudowaliśmy obie wymagane warstwy nawigacji: nawigację stosową (Stack) dla ekranów szczegółowych oraz nawigację zakładkową (Tabs) na dole ekranu.
- **React Native Paper** - biblioteka komponentów UI zgodnych z Material Design. Używaliśmy m.in. Card, Button, TextInput, Searchbar, SnackBar, Badge oraz ActivityIndicator
- **expo-camera** - obsługa aparatu (pierwszy czujnik), wykorzystywana do zrobienia zdjęcia profilowego.
- **expo-location** - pobieranie współrzędnych GPS (drugi czujnik) na ekranie profilu.
- **@react-native-async-storage/async-storage** - lokalne, trwałe przechowywanie danych profilu na urządzeniu.

- **MockAPI** - zewnętrzny, darmowy serwis pełniący rolę backendu REST. Pod adresem zapisanym w constants/api.ts udostępnia zasoby Cars oraz reservations.
- **Jest** wraz z **@testing-library/react-native** - środowisko do testów jednostkowych i komponentowych.
- **ESLint** (konfiguracja eslint-config-expo)- statyczna analiza kodu i pilnowanie spójnego stylu.

Komunikacja z serwerem odbywa się za pomocą wbudowanej funkcji fetch, bez dodatkowej biblioteki typu Axios. Świadomie zrezygnowaliśmy z globalnego magazynu stanu (Redux itp.) przy tej skali aplikacji każdy ekran zarządza swoimi danymi lokalnie (useState), co jest prostsze do utrzymania w małej aplikacji.

2. Implementacja

2.1. Opis głównych funkcjonalności aplikacji

Aplikacja jest podzielona na trzy główne zakładki dolnego paska nawigacji — **Auta**, **Rezerwacje** oraz **Profil**, a z poziomu zakładek otwierają się kolejne ekrany szczegółowe.

Przeglądanie i wyszukiwanie aut. Ekran startowy pobiera listę samochodów (GET /Cars) i wyświetla je jako przewijaną listę kart ze zdjęciem, marką, modelem, rokiem i ceną. Nad listą umieściliśmy pole wyszukiwania, które filtruje auta po marce i nazwie w czasie rzeczywistym bez rozróżniania wielkości liter. Podczas pobierania danych pokazujemy wskaźnik ładowania, a w razie błędu sieci komunikat dla użytkownika.

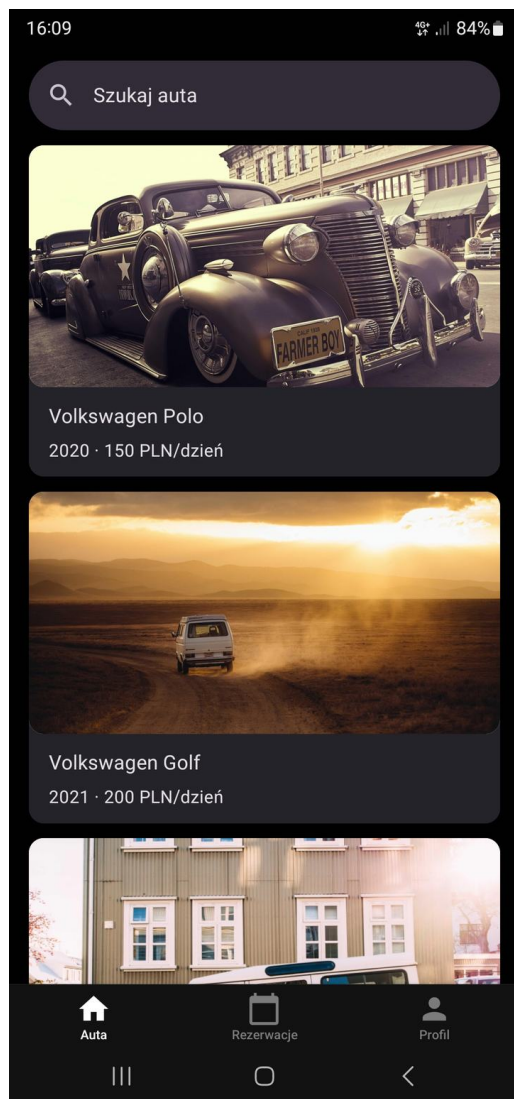
Szczegóły samochodu i rezerwacja. Kliknięcie karty otwiera ekran szczegółów (GET /Cars/:id) z pełnym opisem auta i przyciskiem „Zarezerwuj”. Formularz rezerwacji prosi o datę początkową i końcową. Po ich wpisaniu aplikacja na bieżąco pokazuje liczbę dni i wyliczony koszt, a przycisk potwierdzenia jest aktywny dopiero wtedy, gdy obie daty są uzupełnione. Zatwierdzenie wysyła rezerwację na serwer metodą POST /reservations.

Lista i anulowanie rezerwacji. Zakładka „Rezerwacje” pobiera wszystkie rezerwacje (GET /reservations) i pokazuje je wraz z kolorowym oznaczeniem statusu (zielona „Aktywna”, czerwona „Anulowana”). Listę można odświeżyć gestem pociągnięcia w dół (pull-to-refresh). W szczegółach rezerwacji aktywną rezerwację można anulować, wysyłamy wtedy PUT /reservations/:id ze zmienionym statusem, a użytkownik dostaje potwierdzenie w postaci komunikatu typu „Snackbar”.

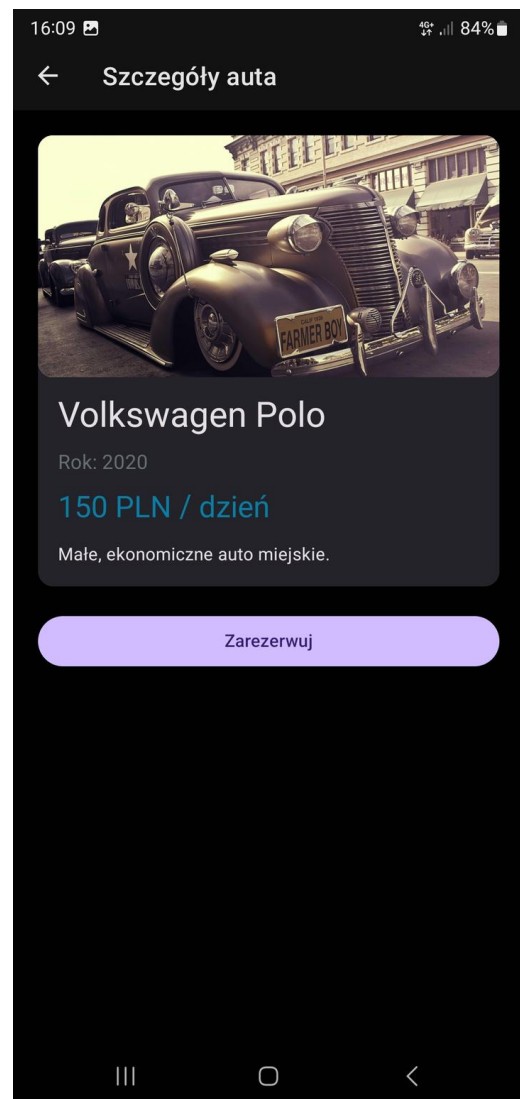
Profil użytkownika z czujnikami. Na zakładce „Profil” prezentujemy dane zapisane lokalnie (imię, e-mail, zdjęcie). Edycja odbywa się na osobnym ekranie, z którego można uruchomić aparat i zrobić zdjęcie profilowe, jest to wykorzystanie pierwszego czujnika. Drugim czujnikiem jest moduł GPS: po naciśnięciu przycisku aplikacja prosi o uprawnienia, pobiera bieżące współrzędne i pozwala otworzyć je w zewnętrznej aplikacji map.

Funkcjonalność	Metoda HTTP	Endpoint
Lista aut	GET	/Cars
Szczegóły auta	GET	/Cars/:id
Utworzenie rezerwacji	POST	/reservations
Lista rezerwacji	GET	/reservations
Szczegóły rezerwacji	GET	/reservations/:id
Anulowanie rezerwacji	PUT	/reservations/:id

2.2. Prezentacja zrzutów ekranu



Rysunek 1. Ekran główny - lista aut z wyszukiwarką.



Rysunek 2. Szczegóły samochodu z przyciskiem rezerwacji.

16:11 4G+ 83%

← Nowa rezerwacja

Volkswagen Polo

150 PLN / dzień

Data od (RRRR-MM-DD)
2026-08-11

Data do (RRRR-MM-DD)
2026-08-13

Łącznie: 2 dni × 150 PLN = 300 PLN

Potwierdź rezerwację

III ○ <

Rysunek 3. Formularz rezerwacji - wyliczanie liczby dni i ceny.

16:11 4G+ 83%

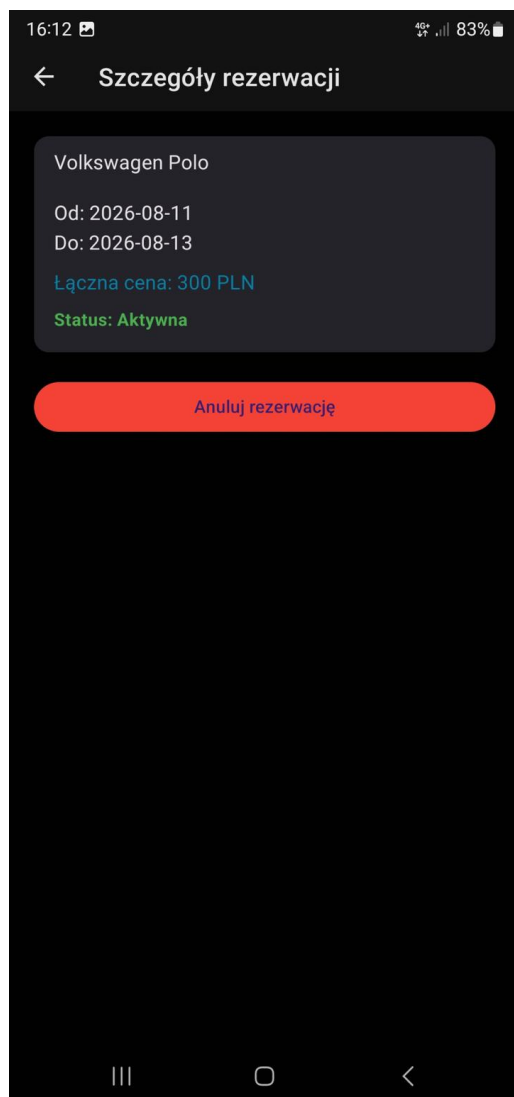
Łącznie: 420 PLN

Volkswagen Polo	2026-06-25 – 2026-06-29	Anulowana	Łącznie: 600 PLN
Volkswagen Golf	2026-05-20 – 2026-05-23	Anulowana	Łącznie: 600 PLN
Renault Clio	2026-06-05 – 2026-06-07	Anulowana	Łącznie: 320 PLN
Skoda Fabia	2026-07-01 – 2026-07-06	Anulowana	Łącznie: 700 PLN
Volkswagen Polo	2025-08-23 – 2025-08-25	Anulowana	Łącznie: 300 PLN
Volkswagen Polo	2026-08-11 – 2026-08-13	Aktywna	Łącznie: 300 PLN

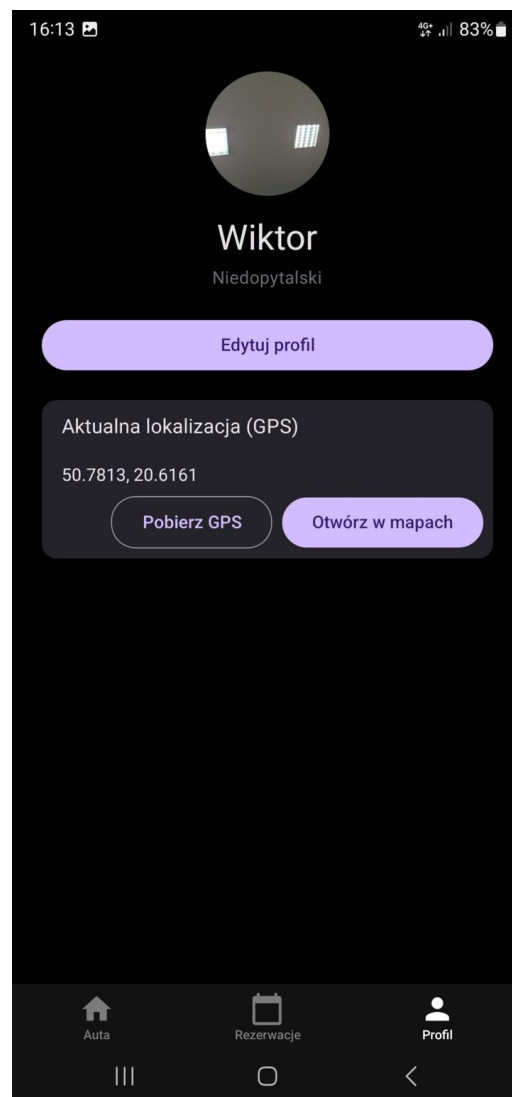
Auta Rezerwacje Profil

III ○ <

Rysunek 4. Lista rezerwacji ze statusami (aktywna / anulowana).



Rysunek 5. Szczegóły rezerwacji i przycisk anulowania.



Rysunek 6. Profil użytkownika - zdjęcie z aparatu oraz lokalizacja GPS.

2.3. Wybrane fragmenty kodu z kluczowymi funkcjonalnościami

Poniżej przedstawiamy kilka fragmentów, które naszym zdaniem najlepiej obrazują sposób działania aplikacji.

Pobieranie i filtrowanie listy aut (app/(tabs)/index.tsx). Dane pobieramy w `useEffect` przy pierwszym renderze, a stan ładowania i błędu trzymamy lokalnie. Filtrowanie odbywa się już po stronie aplikacji, na pobranej tablicy:

```
useEffect(() => {
  fetch(`${API_URL}/Cars`)
    .then((res) => res.json())
    .then((data) => {
      setCars(data);
      setLoading(false);
    })
    .catch(() => {
      setError('Nie można załadować aut.');
```

// Filtrowanie po marce i nazwie (bez rozróżniania wielkości liter)

```
const filtered = cars.filter((car) =>
  `${car.brand} ${car.name}`.toLowerCase().includes(query.toLowerCase())
);
```

Tworzenie rezerwacji i obliczanie ceny (app/reserve/[id].tsx oraz utils/dates.ts). Logikę liczenia dni i kosztu wydzieliliśmy do osobnego pliku pomocniczego, żeby dało się ją łatwo przetestować i żeby nie powtarzać tego samego kodu w kilku miejscach:

```
// utils/dates.ts
export function calculateDays(fromDate: string, toDate: string): number {
  const from = new Date(fromDate);
  const to = new Date(toDate);
  if (isNaN(from.getTime()) || isNaN(to.getTime())) {
    return 0;
  }
  const diff = Math.ceil((to.getTime() - from.getTime()) / (1000 * 60 * 60 * 24));
  return diff > 0 ? diff : 0;
}

export function calculateTotalPrice(fromDate: string, toDate: string, pricePerDay: number): number {
  return calculateDays(fromDate, toDate) * pricePerDay;
}
```

```
// app/reserve/[id].tsx — wysłanie rezerwacji na serwer (POST)
const totalPrice = calculateTotalPrice(fromDate, toDate, car?.price ?? 0);

fetch(`${API_URL}/reservations`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    carId: id,
    carName: `${car?.brand} ${car?.name}`,
    fromDate,
    toDate,
    totalPrice,
    status: 'active',
  }),
})
  .then((res) => res.json())
  .then(() => router.replace('/(tabs)/reservations'));
```


Anulowanie rezerwacji metodą PUT (app/reservation/[id].tsx). Zamiast usuwać rekord, zmieniamy jego status dzięki czemu historia rezerwacji pozostaje widoczna:

```
fetch(`${API_URL}/reservations/${id}`, {
  method: 'PUT',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ status: 'cancelled' }),
})
.then((res) => res.json())
.then((updated) => {
  setReservation(updated);
  setSnackMessage('Rezerwacja anulowana.');
```

Obsługa aparatu (czujnik) (app/camera.tsx). Po zrobieniu zdjęcia wracamy do ekranu edycji profilu i przekazujemy adres URI zdjęcia jako parametr nawigacji, to nasz sposób na przekazanie danych pomiędzy ekranami:

```
const photo = await cameraRef.current.takePictureAsync({ quality: 0.7 });
if (photo) {
  router.dismissTo({
    pathname: '/edit-profile',
    params: { photoUri: photo.uri },
  });
}
```

3. Testy

3.1. Opis metod testowania

Aplikację testowaliśmy dwutorowo: manualnie oraz automatycznie.

Testy manualne prowadziliśmy na bieżąco, uruchamiając aplikację w Expo Go na fizycznym telefonie. Sprawdzaliśmy przede wszystkim pełne ścieżki użytkownika: od wejścia na listę aut, przez utworzenie rezerwacji, aż po jej anulowanie. Osobno testowaliśmy zachowanie aplikacji w sytuacjach „nietypowych”: brak połączenia z serwerem, odmowa udzielenia uprawnień do aparatu lub lokalizacji, a także próba wysłania rezerwacji z niekompletnymi danymi.

Testy automatyczne napisaliśmy w środowisku **Jest** z presetem jest-expo oraz biblioteką **@testing-library/react-native**. Uruchamia się je poleceniem `npm test`. Skupiliśmy się na dwóch obszarach:

- **Testy jednostkowe logiki dat i ceny** (utils/dates.test.ts) sprawdzają funkcje `calculateDays` oraz `calculateTotalPrice`, w tym przypadki brzegowe: poprawny zakres dat, daty sąsiednie, data końcowa równa lub wcześniejsza od początkowej oraz dane nieprawidłowe (puste lub niepoprawne ciągi znaków).
- **Test komponentu listy rezerwacji** (__tests__/ReservationsScreen.test.tsx) renderuje ekran rezerwacji z podstawionym (zamockowanym) `fetch` i weryfikuje dwa scenariusze: wyświetlenie komunikatu o braku rezerwacji dla pustej listy oraz poprawne wyrenderowanie rezerwacji zwróconej z API wraz z łączną ceną.

Wybór tych elementów do testowania nie był przypadkowy. Logika obliczeń jest „czysta” (nie zależy od interfejsu), więc idealnie nadaje się do testów jednostkowych. Z kolei ekran rezerwacji jest dobrym reprezentantem typowego komponentu pobierającego dane z sieci, dlatego posłużył nam do sprawdzenia całej ścieżki: pobranie -> przetworzenie -> wyświetlenie.

3.2. Wyniki testów, napotkane błędy oraz zastosowane rozwiązania

Wszystkie testy automatyczne przechodzą poprawnie. Aktualny wynik z konsoli wygląda następująco:

```
Test Suites: 2 passed, 2 total
Tests:      9 passed, 9 total
Snapshots:  0 total
```

W trakcie pisania testów natrafił się na kilka problemów, które warto opisać:

- 1. Mockowanie nawigacji i hooka `useFocusEffect`.** Ekran rezerwacji korzysta z `useFocusEffect` z Expo Router, który w środowisku testowym nie ma realnego kontekstu nawigacji. Test zatrzymywał się na błędzie importu. Rozwiązaliśmy to, podstawiając w teście własną, uproszczoną implementację `expo-router (jest.mock)`, w której `useFocusEffect` wywołuje przekazaną funkcję wewnątrz zwykłego `useEffect`.
- 2. Transformacja bibliotek z `node_modules`.** Domyślnie Jest nie przetwarza kodu z `node_modules`, przez co biblioteki takie jak React Native Paper powodowały błędy składni przy imporcie. Musieliśmy dopisać `react-native-paper` (oraz pozostałe pakiety RN/Expo) do `transformIgnorePatterns` w konfiguracji Jest w `package.json`.
- 3. Ostrzeżenie `act(...)` przy odświeżaniu listy.** Po dodaniu odświeżania danych w `useFocusEffect` w konsoli testów pojawia się ostrzeżenie, że aktualizacja stanu nie jest opakowana w `act(...)`. Jest to znane ostrzeżenie wynikające z asynchronicznego ustawiania stanu po zakończeniu zapytania, nie powoduje ono jednak błędu, a wszystkie asercje testów przechodzą (korzystamy z `waitFor`, który czeka na ustabilizowanie się komponentu). Zostawiliśmy to jako znany, nieblokujący komunikat.

Po stronie testów manualnych największą wartością okazało się wychwycenie sytuacji, w której użytkownik mógł zatwierdzić rezerwację bez podania dat. Dodaliśmy zabezpieczenie blokujące przycisk „Potwierdź rezerwację” do momentu uzupełnienia obu pól, co wyeliminowało wysyłanie niekompletnych rekordów na serwer.

4. Podsumowanie

4.1. Wnioski z realizacji projektu

Udało nam się zrealizować wszystkie założenia postawione na początku. Aplikacja zawiera dwa rodzaje nawigacji (stosową i zakładkową), korzysta z dwóch czujników (aparat i GPS) oraz komunikuje się z zewnętrznym API trzema metodami (GET, POST, PUT).

Najważniejszym wnioskiem jest dla nas to, jak bardzo dobra struktura projektu ułatwia późniejszą pracę. Konsekwentne trzymanie się wzorca „każdy ekran pobiera swoje dane i zarządza swoim stanem” sprawiło, że dodawanie kolejnych funkcji było przewidywalne i nie wymagało przebudowy całości. Podobnie wydzielenie logiki obliczeń do osobnego modułu opłaciło się przy testach.

4.2. Ocena osiągniętych rezultatów i refleksje na temat procesu implementacji

Z efektu jesteśmy zadowoleni. Aplikacja działa płynnie, interfejs jest czytelny i spójny dzięki React Native Paper, a obsługa błędów (komunikaty, wskaźniki ładowania) sprawia, że korzystanie z niej jest przyjemne nawet przy wolniejszym połączeniu.

Patrząc wstecz, kilka rzeczy zrobilibyśmy inaczej. Na początku część zapytań sieciowych pisaliśmy „od zera” w każdym ekranie, przez co powtarzał się ten sam kod. Gdybyśmy zaczęli od nowa, prawdopodobnie od razu przygotowalibyśmy jeden wspólny moduł do obsługi API. Sporo czasu zajęło nam też skonfigurowanie środowiska testowego pod Expo. Samo napisanie testów było łatwiejsze niż doprowadzenie Jest do tego, żeby w ogóle uruchomił komponenty React Native.

TypeScript w trybie strict na początku bywał uciążliwy, lecz finalnie kilkakrotnie uchronił nas przed błędami związanymi z brakującymi lub źle nazwanymi polami danych. Z perspektywy całego projektu uważamy ten wybór za słuszny.

4.3. Propozycje usprawnień lub dalszego rozwoju aplikacji

Aplikacja w obecnej formie jest funkcjonalnie kompletna, ale widzimy kilka naturalnych kierunków rozwoju:

- **Logowanie i konta użytkowników:** obecnie profil jest lokalny, pełne uwierzytelnianie pozwoliłoby powiązać rezerwacje z konkretnym kontem.
- **Filtrowanie i sortowanie aut:** np. po cenie, marce lub roku produkcji, co byłoby przydatne przy większej liczbie samochodów.
- **Powiadomienia push:** np. przypomnienie o zbliżającym się terminie rozpoczęcia wynajmu.

5. Podział pracy

Projekt realizowaliśmy w dwuosobowym zespole, dzieląc się zadaniami tak, aby każdy odpowiadał za spójny obszar aplikacji, ale jednocześnie konsultowaliśmy ze sobą najważniejsze decyzje projektowe.

Osoba	Zakres odpowiedzialności
Wiktor Niedopytalski	Konfiguracja projektu Expo, struktura nawigacji (Stack + Tabs), integracja z API (auta i rezerwacje), obsługa czujników (aparat, GPS), testy automatyczne, logika rezerwacji i obliczeń.
Kamil Grabiec	Część ekranów i komponentów interfejsu (React Native Paper), ekran profilu i lokalne przechowywanie danych, konfiguracja API, przygotowanie dokumentacji i sprawozdania.

W praktyce granice te bywały płynne, przy trudniejszych fragmentach (np. konfiguracja środowiska testowego) pracowaliśmy wspólnie. Kod rozwijaliśmy w repozytorium Git, co ułatwiało łączenie naszych zmian i śledzenie historii projektu.

6. Literatura

1. Dokumentacja Expo - <https://docs.expo.dev/>
2. Dokumentacja React Native - <https://reactnative.dev/docs/getting-started>
3. Expo Router (routing plikowy) - <https://docs.expo.dev/router/introduction/>
4. React Native Paper - <https://callstack.github.io/react-native-paper/>
5. Expo Camera - <https://docs.expo.dev/versions/latest/sdk/camera/>
6. Expo Location - <https://docs.expo.dev/versions/latest/sdk/location/>
7. Async Storage - <https://react-native-async-storage.github.io/async-storage/>
8. MockAPI - <https://mockapi.io/>
9. Jest - <https://jestjs.io/docs/getting-started>
10. Testing Library (React Native) - <https://callstack.github.io/react-native-testing-library/>
11. Dokumentacja języka TypeScript - <https://www.typescriptlang.org/docs/>